

Tic Tac Toe Game

Comprehensive Code Documentation & Architecture Explanation

1. OVERVIEW

This project is a CLI-based implementation of the classic 2-player **Tic Tac Toe** game written in standard Python 3. It provides a simple, interactive terminal UI with full input validation, automated win/tie detection, and dynamic replay capabilities.

2. CORE FEATURES

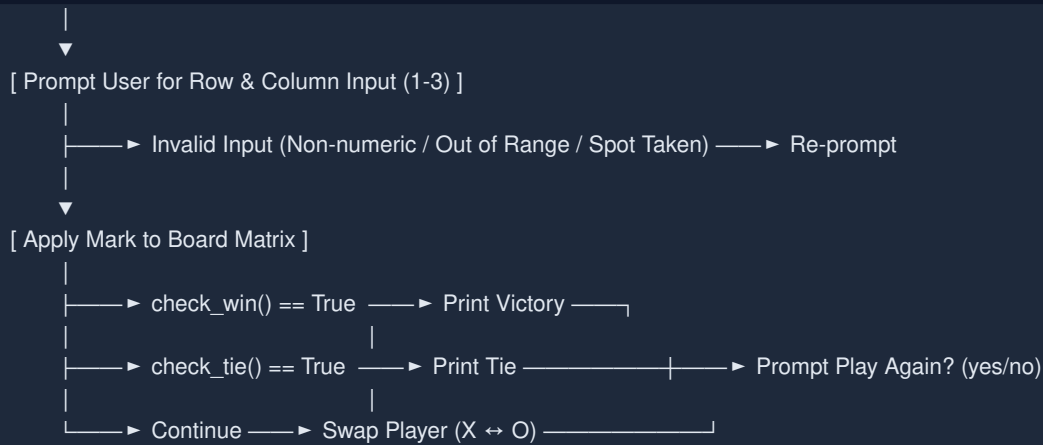
- Grid Rendering** Custom formatted ASCII grid displaying move state after every turn.
- Input Offset Handling** Maps 1-indexed user input (1-3) directly to 0-indexed matrix indices (0-2).
- Input Validation** Handles non-numeric strings, out-of-bounds selections, and already-occupied cells without throwing uncaught exceptions.
- State Evaluation** Evaluates horizontal rows, vertical columns, and both diagonals to determine win or tie states after each turn.
- Replay Loop** Prompts players at the end of each game to initiate a new board state or exit.

3. ARCHITECTURE & FUNCTION BREAKDOWN

FUNCTION NAME	PRIMARY RESPONSIBILITY	KEY LOGIC / IMPLEMENTATION
<code>print_board(board)</code>	Renders the current 3x3 game board in the console.	Iterates through rows using f-string formatting with vertical separators (<code> </code>) and horizontal borders (<code>---</code> <code>---</code> <code>---</code>).
<code>check_win(board, player)</code>	Determines if the current player has formed a winning line.	Evaluates row/column iterations via <code>all()</code> checks, alongside explicit conditions for main and anti-diagonals.
<code>check_tie(board)</code>	Checks if all spaces are filled without a winner.	Scans every row matrix array for empty space characters (<code>" "</code>).
<code>play_game()</code>	Manages main execution loop, turn swapping, and user interactions.	Initializes dynamic board state, coordinates input validation, checks game state triggers, and manages overall game replay.

4. WORKFLOW & STATE FLOW

```
[ Start Game Loop ]
  |
  ▼
[ Initialize 3x3 Board & Set Player = "X" ]
  |
  ▼
[ Render Board via print_board() ]
```



5. DETAILED CODE COMPONENT ANALYSIS

5.1 Board Initialization & User-Friendly Coordinates

The game board is stored in memory as a nested list (2D matrix) initialized with empty space strings:

```
board = [["_ " for _ in range(3)] for _ in range(3)]
```

To make input intuitive for non-programmers, user inputs are decremented by 1:

```
row = int(input("Enter row (1-3): ")) - 1
col = int(input("Enter column (1-3): ")) - 1
```

5.2 Victory Logic (`check_win`)

The function evaluates winning configurations cleanly:

```
# Checks all 3 rows and all 3 columns using generator expressions
for i in range(3):
    if all(board[i][j] == player for j in range(3)) or \
       all(board[j][i] == player for j in range(3)):
        return True

# Checks main diagonal (top-left to bottom-right) and anti-diagonal (top-right to bottom-left)
if (board[0][0] == player and board[1][1] == player and board[2][2] == player) or \
    (board[0][2] == player and board[1][1] == player and board[2][0] == player):
    return True
```

6. HOW TO RUN

```
python main.py
```